M.SaiRam
RegNo: 192425291

Assessment-3

UNIT – III

PROBLEM 1: FULL-STACK WEB APP DEVELOPMENT FOR AN E-COMMERCE DASHBOARD

1.1 Introduction

An e-commerce analytics dashboard collects sales, customer, product, and marketing data and presents it through interactive charts and reports.

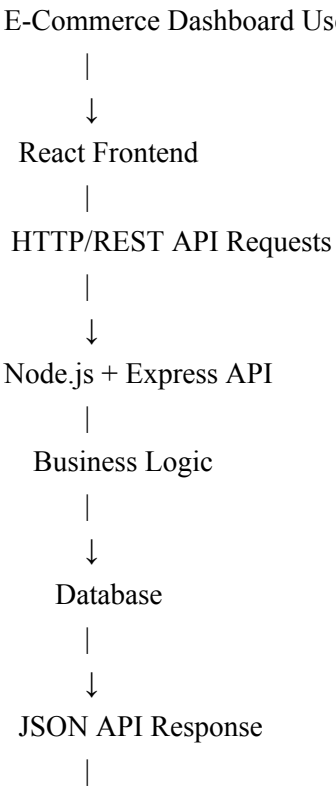
The proposed system uses React for the frontend, Node.js and Express for the backend API, and a database for storing application data. The system follows a Git-based development workflow with pull requests and code reviews.

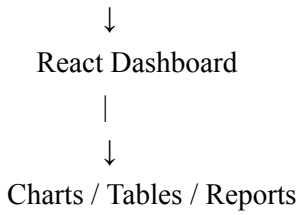
1.2 System Architecture

The main components of the system are:

1. React Frontend – Provides the user interface and interactive charts.
2. Node.js/Express Backend – Provides REST APIs and handles business logic.
3. Database – Stores sales, customer, product, and analytics information.
4. Git Repository – Manages source code and collaboration.
5. CI Server – Performs automated testing and code quality checks.

System Architecture Diagram:





1.3 React Frontend

The React application contains reusable components such as:

• Navigation Bar
• Dashboard
• Sales Chart
• Revenue Chart
• Product Table
• Customer Statistics
• Date Filter
• Loading and Error Components

React communicates with the backend using HTTP requests.

For example:

GET /api/sales

The backend returns data in JSON format, which React uses to update charts and tables.

1.4 Node.js and Express Backend

The backend is responsible for:

• Handling API requests.
• Validating user input.
• Performing business logic.
• Communicating with the database.
• Returning JSON responses.

Example API endpoints:

GET /api/sales
GET /api/products
GET /api/customers
GET /api/orders
GET /api/analytics

1.5 API Data Flow

The data flow occurs as follows:

1. The user opens the React dashboard.
2. React sends an HTTP request to the Express API.
3. Express receives the request.
4. The backend validates the request.

5. Express queries the database.
6. The database returns the required information.
7. Express converts the data into JSON.
8. The API sends the JSON response to React.
9. React updates the charts and tables.

Data Flow Diagram:

User
↓
React Component
↓
HTTP Request
↓
Express API
↓
Business Logic
↓
Database
↓
JSON Response
↓
React State
↓
Chart / Table

1.6 Real-Time Data

For frequently changing dashboard information, the application can use polling or WebSocket communication.

For example, the frontend can request updated sales information every few seconds.

For highly interactive real-time dashboards, WebSockets can be used so that the server can send updates directly to connected clients.

1.7 Git Branching Strategy

The recommended approach is Feature Branching.

Main branches:

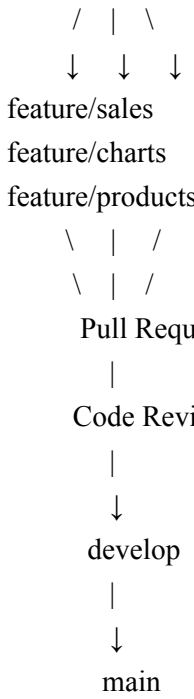
main – Contains stable production-ready code.

develop – Contains integrated development code.

feature branches – Used for individual features.

Git Workflow:

```
        main
          |
          ↓
        develop
       /   |   \
```



Example:
feature/sales-dashboard
feature/product-chart
feature/customer-report
Developers should never directly push unfinished code to the main branch.
1.8 CI Flow
The Continuous Integration process should be:
Developer creates feature branch
↓

Developer writes code
↓

Push code to Git repository
↓

Create Pull Request
↓

CI Pipeline Starts
↓

Install Dependencies

↓

Linting

↓

Unit Tests

↓

Build Application

↓

Code Review

↓

Merge

CI checks should include:

• Dependency installation
• ESLint or equivalent linting
• Unit tests
• Build verification
• API tests
• Security/dependency checks

1.9 Pull Request Review Checklist

Before approving a pull request, reviewers should check:

[ ] The code solves the required problem.
[ ] The code follows the project coding standards.
[ ] No unnecessary code has been added.
[ ] Variable and function names are meaningful.
[ ] Unit tests have been added or updated.
[ ] Existing tests are passing.
[ ] No sensitive information such as passwords or API keys is committed.
[ ] API changes are properly documented.
[ ] UI changes work correctly on different screen sizes.
[ ] The application builds successfully.
[ ] No major performance issues are introduced.
[ ] The code is easy to understand and maintain.

1.10 Conclusion

The proposed React, Node.js, Express, and database architecture provides a clear separation between frontend, backend, and data storage. Git feature branching, pull requests, code reviews, and CI checks improve collaboration and software quality. The architecture can also support real-time dashboard updates and future scalability.

PROBLEM 2: DOCKERIZING A NEWS AGGREGATOR APPLICATION

2.1 Introduction

A news aggregator collects news articles from multiple APIs and provides them through a common application. Docker can package the backend application and its dependencies into a portable container.

This ensures that the application runs consistently on the developer's computer, CI server, testing environment, and production server.

2.2 Dockerfile

The following Dockerfile can be used for a Node.js backend application:

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 3000
CMD ["npm", "start"]
```

2.3 Explanation of Dockerfile Commands

FROM node:20-alpine

This selects a lightweight Node.js image as the base image.

WORKDIR /app

This creates and selects /app as the working directory inside the container.

COPY package*.json ./

This copies the package.json and package-lock.json files into the container.

RUN npm install

This installs the Node.js dependencies.

COPY . .

This copies the application source code into the container.

EXPOSE 3000

This indicates that the application uses port 3000.

CMD ["npm", "start"]

This starts the Node.js application when the container runs.

2.4 Building the Docker Image

Open a terminal in the project directory and execute:

docker build -t news-aggregator:1.0 .

The command creates a Docker image called news-aggregator with version 1.0.

2.5 Running the Container

Execute:

docker run -d -p 3000:3000 --name news-app news-aggregator:1.0

Explanation:

-d runs the container in the background.

-p 3000:3000 maps the host port to the container port.

--name news-app gives the container a name.

news-aggregator:1.0 is the Docker image.

The application can then be accessed using:

http://localhost:3000

2.6 Checking Running Containers

Command:

docker ps

To view application logs:

docker logs news-app

To stop the container:

docker stop news-app

2.7 Pushing the Image to Docker Hub

Step 1: Log in to Docker Hub.

docker login

Step 2: Tag the image.

docker tag news-aggregator:1.0 USERNAME/news-aggregator:1.0

Step 3: Push the image.

docker push USERNAME/news-aggregator:1.0

The image can then be downloaded on another machine using:

docker pull USERNAME/news-aggregator:1.0

Step 4: Run the downloaded image.

docker run -d -p 3000:3000 USERNAME/news-aggregator:1.0

2.8 Why Containerization Improves Consistency

Without Docker, an application may behave differently because different machines have different:

• Operating systems

• Node.js versions

• Package versions

• Environment configurations

• System dependencies

Docker packages the application and its dependencies together.

Therefore:

Developer Machine

↓

Same Docker Image

↓

CI Server
↓
Same Docker Image
↓
Production Server

The same container image can be used across all environments, reducing the "works on my machine" problem.

2.9 Conclusion

Docker provides portability, consistency, and easy deployment. By packaging the news aggregator backend with its dependencies, the application can run reliably across development, testing, CI, and production environments.

PROBLEM 3: KUBERNETES DEPLOYMENT FOR A VIDEO-STREAMING MICROSERVICE

3.1 Introduction

A video-streaming platform may receive a very large number of requests when popular content is being watched. Kubernetes can automatically manage multiple instances of the Video API and provide scaling and fault tolerance.

The proposed solution uses:

• Kubernetes Deployment
• Multiple Pods
• Kubernetes Service
• Horizontal Pod Autoscaler
• Helm for packaging

3.2 Kubernetes Deployment YAML

The following Deployment creates three replicas of the Video API:

```
apiVersion: apps/v1
kind: Deployment
metadata:
name: video-api
spec:
replicas: 3
selector:
matchLabels:
app: video-api
template:
metadata:
labels:
app: video-api
spec:
containers:
- name: video-api
image: myregistry/video-api:1.0
ports:
```

- containerPort: 8080
resources:
requests:
cpu: "250m"
memory: "256Mi"
limits:
cpu: "500m"
memory: "512Mi"

3.3 Explanation of Deployment

The Deployment manages the Video API Pods.

replicas: 3

This creates three running instances of the application.

If one Pod fails, Kubernetes can create another Pod to maintain the desired number of replicas.

The containerPort 8080 indicates that the application listens on port 8080.

3.4 Horizontal Pod Autoscaler

The Horizontal Pod Autoscaler automatically increases or decreases the number of Pods based on resource usage.

Example:

apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
name: video-api-hpa
spec:
scaleTargetRef:
apiVersion: apps/v1
kind: Deployment
name: video-api
minReplicas: 3
maxReplicas: 10
metrics:
- type: Resource
resource:
name: cpu
target:
type: Utilization
averageUtilization: 70

Explanation:

Minimum Pods = 3

Maximum Pods = 10

Target CPU utilization = 70%

If CPU usage increases significantly, Kubernetes can increase the number of Pods.

Example:

Normal Traffic:

3 Pods

High Traffic:

3 → 5 → 7 → 10 Pods

When traffic decreases, Kubernetes can reduce the number of Pods within the configured limits.

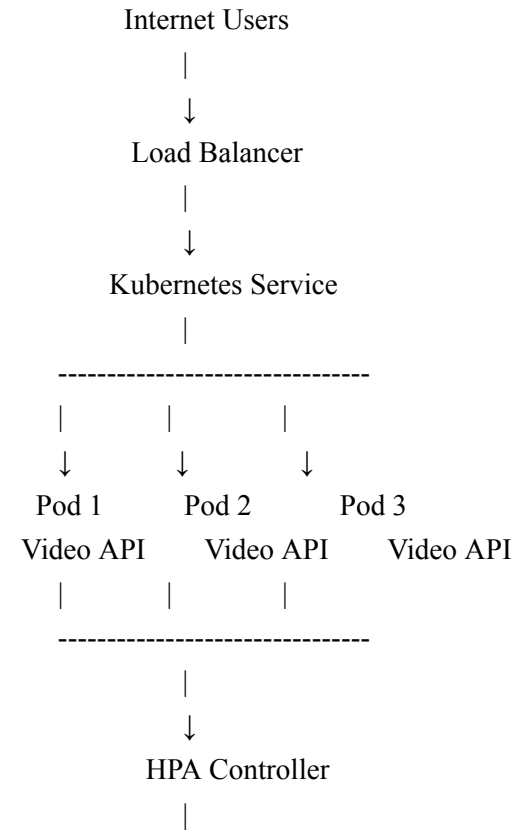
3.5 Kubernetes Service YAML

The Service provides a stable network endpoint for the Video API.

```
apiVersion: v1
kind: Service
metadata:
name: video-api-service
spec:
selector:
app: video-api
ports:
- protocol: TCP
port: 80
targetPort: 8080
type: LoadBalancer
```

The LoadBalancer type allows external clients to access the service through a cloud load balancer.

3.6 Kubernetes Architecture



CPU / Resource Usage

|

↓

Create or Remove Pods

3.7 High-Level Helm Chart Structure

A Helm chart can be organized as follows:

video-api/

│

├── Chart.yaml

├── values.yaml

├── README.md

│

└── templates/

├── deployment.yaml

├── service.yaml

├── hpa.yaml

└── configmap.yaml

3.8 Explanation of Helm Files

Chart.yaml

Contains chart metadata such as application name and version.

values.yaml

Contains configurable values such as:

• Docker image

• Replica count

• CPU limits

• Memory limits

• Service type

templates/deployment.yaml

Contains the Kubernetes Deployment template.

templates/service.yaml

Contains the Kubernetes Service template.

templates/hpa.yaml

Contains the autoscaling configuration.

templates/configmap.yaml

Contains non-sensitive application configuration.

3.9 Benefits of Kubernetes

1. Automatic scaling.
2. Self-healing when Pods fail.
3. Load balancing.

4. Rolling deployments.
5. Efficient resource management.
6. High availability.

3.10 Conclusion

Kubernetes is suitable for the Video API because it can maintain multiple application instances, automatically scale according to demand, and recover from Pod failures. Helm further simplifies deployment and configuration management.

PROBLEM 4: GIT COLLABORATION FOR A SOCIAL MEDIA APP

4.1 Introduction

A social media application developed by six developers requires a controlled Git workflow. Directly pushing code to the main branch can result in broken builds, conflicts, and bugs.

A branching strategy with pull requests, mandatory code reviews, and automated CI checks can improve software quality.

4.2 Recommended Git Workflow

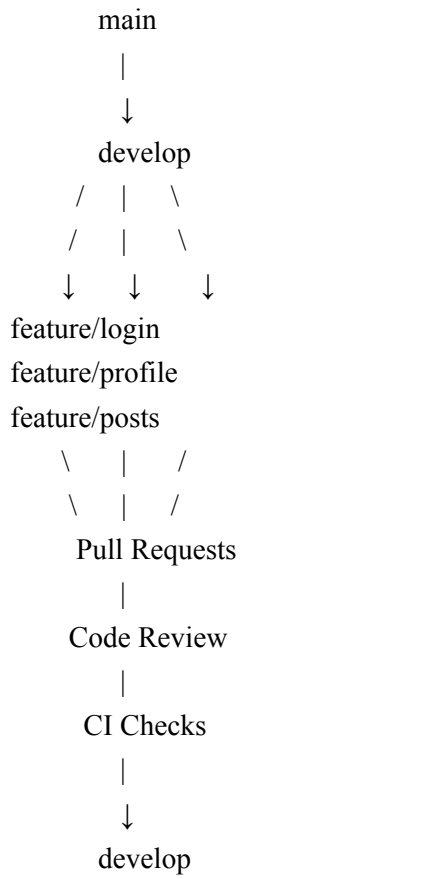
The recommended workflow contains:

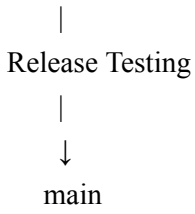
main – Production-ready code.

develop – Integration branch for development.

feature branches – Individual development branches.

Workflow Diagram:





4.3 Feature Branch Naming

Developers should use meaningful branch names.

Examples:

feature/user-login

feature/create-post

feature/comment-system

feature/notification

feature/profile-page

bugfix/login-error

4.4 Pull Request Rules

The following rules should be enforced:

1. Developers must not directly push to main.
2. Developers must create a feature branch for new work.
3. Every completed feature must be submitted through a Pull Request.
4. At least one or two developers should review important changes.
5. All CI checks must pass before merging.
6. The Pull Request should have a clear title and description.
7. Large and unrelated changes should not be combined into one Pull Request.
8. Review comments should be resolved before merging.
9. The branch should be updated with the latest develop changes when necessary.
10. The Pull Request should be merged only after approval.

4.5 Code Review Checklist

Reviewers should verify:

[ ] Code follows project standards.

[ ] Code is readable and maintainable.

[ ] No unnecessary code is included.

[ ] Unit tests are present.

[ ] Existing functionality is not broken.

[ ] Security issues are not introduced.

[ ] No passwords or API keys are committed.

[ ] Error handling is implemented.

[ ] Documentation is updated when required.

4.6 CI Checks Before Merge

The CI pipeline should automatically perform:

1. Dependency installation.
2. Code linting.
3. Unit testing.
4. Integration testing where required.
5. Build verification.
6. Security/dependency scanning.

Example CI flow:

Pull Request
↓
Install Dependencies
↓
Lint
↓
Unit Tests
↓
Build
↓
Security Check
↓
All Checks Pass
↓
Code Review
↓
Merge

If any required check fails, the Pull Request should not be merged.

4.7 Guidelines for Resolving Merge Conflicts

Step 1:

Update the local develop branch.

git checkout develop

git pull origin develop

Step 2:

Switch to the feature branch.

git checkout feature/create-post

Step 3:

Merge or rebase the latest develop changes.

git merge develop

Step 4:

Git identifies conflicting files.

Open the files and resolve the conflict manually.

Step 5:

Add the resolved files.

git add .

Step 6:

Commit the resolution.

git commit -m "Resolve merge conflicts"

Step 7:

Push the updated branch.

git push origin feature/create-post

Step 8:

Run all tests again before updating the Pull Request.

4.8 Good Collaboration Practices

• Communicate before making major changes.

• Keep commits small and meaningful.

• Pull the latest changes regularly.

• Avoid modifying unrelated files.

• Use clear commit messages.

• Review other developers' Pull Requests.

• Do not commit generated files unnecessarily.

• Resolve conflicts early rather than at the end of the project.

4.9 Conclusion

A structured Git workflow prevents developers from directly modifying production code. Feature branches, Pull Requests, mandatory reviews, and CI checks provide a reliable development process and reduce bugs and merge conflicts.

PROBLEM 5: HIGH-PERFORMANCE ALGORITHM OPTIMIZATION FOR A RIDE-SHARING PLATFORM

5.1 Introduction

A ride-sharing platform may receive more than 100,000 ride requests per minute during peak periods. The matching system must quickly find suitable drivers for riders.

A simple algorithm that checks every driver for every rider becomes inefficient as the number of users increases.

Therefore, efficient data structures, spatial indexing, caching, asynchronous processing, and distributed computing should be used.

5.2 Identification of Algorithm Bottlenecks

The major bottlenecks are:

1. Checking Every Driver

If every ride request checks every available driver, the algorithm can approach $O(R \times D)$, where R is the number of ride requests and D is the number of available drivers.

This becomes extremely slow at large scale.

2. Repeated Distance Calculations

Calculating the exact distance between a rider and thousands of drivers for every request consumes significant CPU resources.

3. Database Queries

Frequently querying a central database for nearby drivers creates high database load and increases response time.

4. Lock Contention

Multiple worker threads trying to update the same driver information can cause contention.

5. Network Communication

A centralized matching service may become overloaded when millions of requests are processed.

5.3 Proposed Optimized Algorithm

The system should use a spatial data structure such as a Geohash-based index, grid index, or spatial database index.

Instead of searching every driver, drivers are grouped according to geographical regions.

Example:

City
|
+---- Zone A
| |
| +---- Drivers
|
+---- Zone B
| |
| +---- Drivers
|
+---- Zone C
|
+---- Drivers

When a rider requests a vehicle, the system first identifies the rider's geographical zone.

It then searches only the current zone and nearby zones.

5.4 Optimized Matching Steps

Step 1:

Receive the rider request containing:

• Rider location

• Destination

• Vehicle type

• Other preferences

Step 2:

Convert the rider's location into a spatial cell or Geohash.

Step 3:

Retrieve available drivers from the same cell.

Step 4:

If not enough drivers are available, search neighboring cells.

Step 5:

Filter drivers based on:

• Availability

• Vehicle type

• Distance

• Driver status

Step 6:

Calculate a matching score.

Example scoring formula:

Matching Score =

Distance Score + ETA Score + Driver Rating Score + Availability Score

Step 7:

Select the best available driver.

Step 8:

Reserve the driver using an atomic operation.

Step 9:

Send the ride assignment to the driver.

5.5 Why the Optimized Algorithm is Faster

A naive algorithm searches all drivers.

For example:

100,000 drivers

1 ride request

Naive approach:

Check approximately 100,000 drivers.

With spatial indexing:

Only drivers in the rider's geographical area are considered.

For example:

100,000 total drivers

Approximately 100–500 nearby candidates

Therefore, the amount of computation is greatly reduced.

5.6 Data Structures

Suitable data structures include:

1. Hash Map

Used for fast lookup of driver information.

Average lookup time is approximately O(1).

2. Spatial Grid

Divides geographical regions into cells and stores nearby drivers in each cell.

3. Geohash

Converts geographical coordinates into strings representing geographical areas. Drivers with similar Geohashes are generally geographically close.

4. Priority Queue

Can be used to maintain the nearest or highest-scoring candidates.

5. Cache

Frequently accessed driver availability and location data can be stored in a fast in-memory cache.

5.7 Caching

A distributed cache such as Redis can store:

• Driver location
• Driver availability
• Vehicle type
• Driver status
• Temporary ride information

Instead of repeatedly querying the database:

Request
↓
Cache
↓
Nearby Drivers

Only persistent information needs to be stored in the main database.

This reduces database load and improves response time.

5.8 Concurrency Model

The matching system should use asynchronous and concurrent processing.

Different technologies can be used depending on the programming language and architecture.

Threads

Threads can execute multiple independent tasks concurrently.

For example:

Thread 1 → Zone A matching

Thread 2 → Zone B matching

Thread 3 → Zone C matching

Async Processing

Asynchronous programming allows the system to process network requests without blocking the main execution flow.

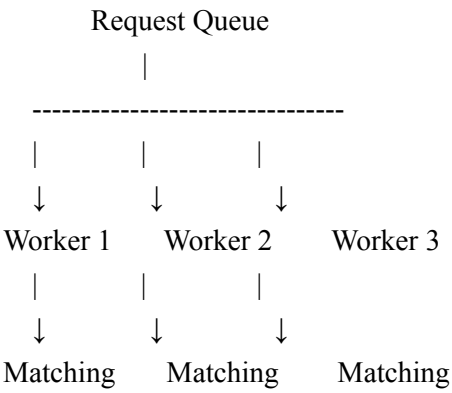
It is useful for:

• Database operations
• Cache operations
• API calls
• Network communication

Worker Processes

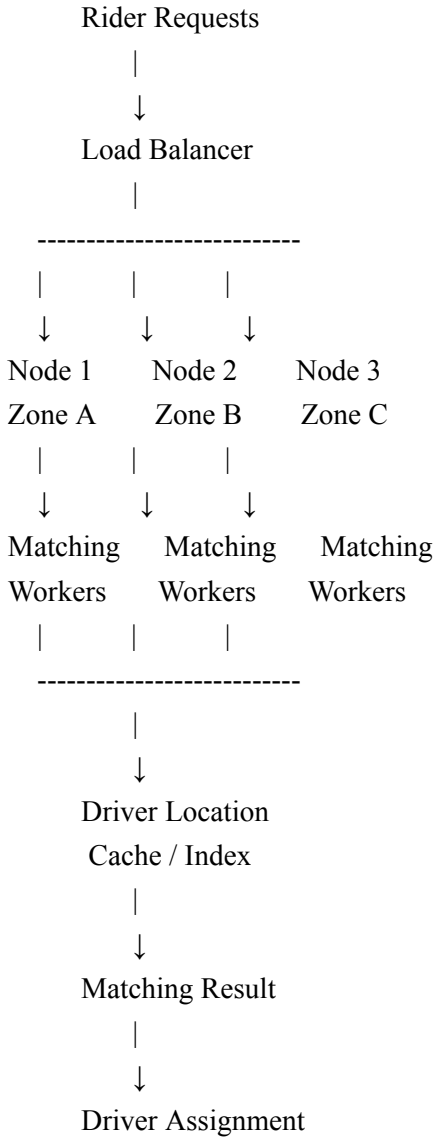
A pool of workers can process ride requests in parallel.

Example:



5.9 Distributed Matching Architecture

The system should distribute ride requests across multiple compute nodes.



5.10 Distributed Matching Workflow

1. Rider sends a ride request.
2. Load balancer receives the request.
3. The request is routed to an appropriate matching node.
4. The matching node identifies the geographical zone.
5. The node retrieves nearby drivers from the cache or spatial index.
6. Candidate drivers are filtered.
7. Matching scores are calculated.
8. The best driver is selected.
9. The driver is atomically reserved.
10. The ride assignment is sent to the driver.
11. The matching result is stored for tracking and analytics.

5.11 Handling High Traffic

When traffic increases:

Normal Traffic:

Node 1 + Node 2 + Node 3

High Traffic:

Node 1 + Node 2 + Node 3 + Node 4 + Node 5 + Node 6

Additional workers or compute nodes can be added horizontally.

This prevents a single server from becoming a bottleneck.

5.12 Reliability Considerations

The matching system should also include:

• Retry mechanisms.
• Request timeouts.
• Idempotency.
• Health checks.
• Load balancing.
• Circuit breakers.
• Monitoring and logging.
• Driver reservation using atomic operations.

Atomic reservation is especially important because two riders should not be assigned the same driver at the same time.

5.13 Complexity Comparison

Naive Approach:

For each rider, search every available driver.

Approximate complexity:

$O(R \times D)$

Optimized Spatial Approach:

Search only nearby spatial cells.

The effective number of candidates is much smaller than D, making the matching process substantially faster in practice.

5.14 Conclusion

A ride-sharing platform cannot rely on a simple linear search when processing more than 100,000 requests per minute. Spatial indexing reduces the search area, caching reduces database access, priority queues help select candidates efficiently, and concurrency allows multiple requests to be processed simultaneously.

A distributed architecture with multiple matching nodes, load balancing, and worker processes provides scalability and high availability. These techniques significantly improve the speed and reliability of large-scale ride-matching systems.

OVERALL CONCLUSION

Full-stack development, containerization, Kubernetes, Git collaboration, and advanced algorithms are important technologies in modern software engineering.

In Problem 1, React, Node.js, Express, and database integration provide a complete full-stack architecture. Feature branching, Pull Requests, code reviews, and CI improve development quality.

In Problem 2, Docker provides a consistent environment for the news aggregator application and allows the same image to be deployed across development, testing, CI, and production environments.

In Problem 3, Kubernetes provides container orchestration, automatic scaling, service exposure, and fault tolerance for the Video API. Helm simplifies deployment and configuration management.

In Problem 4, Git feature branches, Pull Requests, mandatory reviews, and CI checks provide an organized workflow for a six-member development team.

In Problem 5, spatial indexing, caching, concurrency, and distributed computing provide an efficient solution for large-scale ride matching.

Therefore, these technologies and techniques help software teams build applications that are scalable, reliable, maintainable, efficient, and suitable for real-world production environments.